МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №6
по курсу «Дискретный анализ»
на тему «Динамическое программирование»

Выполнил: Н. В. Храмов
Группа: М8О-208БВ-24
Преподаватель: Макаров Н.К.
Проверил: Сахарин Н.А.
Дата: 02.10.2026
Оценка:
Подпись:

Москва, 2026

Постановка задачи

Дано натуральное число n. За один ход разрешается вычесть единицу, разделить число на два или разделить его на три. Деление допускается только нацело. Стоимость каждой операции равна текущему значению числа до её выполнения.

Необходимо преобразовать число n в единицу так, чтобы суммарная стоимость всех операций была минимальной, и вывести одну из оптимальных последовательностей операций.

Формат ввода

В первой строке задано целое число n, $2 \le n \le 10^7$.

Формат вывода

В первой строке выводится минимальная стоимость преобразования. Во второй строке — последовательность операций, разделённых пробелами: -1, /2 или /3. При наличии нескольких оптимальных последовательностей подходит любая из них.

Алгоритм решения задачи

Решение основано на динамическом программировании. Обозначим через dp[i] минимальную стоимость преобразования числа i в единицу. Для единицы операции не нужны, поэтому dp[1] = 0.

Для каждого i от 2 до n рассматриваются все допустимые первые операции. Их стоимость одинакова и равна i, поэтому достаточно выбрать переход с наименьшей стоимостью оставшегося преобразования.

$$dp[i] = i + \min \{ dp[j] : j \in S(i) \},$$
$$S(i) = \{i - 1\} \cup \{i / 2, \text{если } i \text{ кратно } 2\} \cup \{i / 3, \text{если } i \text{ кратно } 3\}.$$

Этапы решения

1. Заполнение таблицы. Значения dp вычисляются по возрастанию i. Для каждой позиции в массиве op сохраняется операция, которая обеспечивает минимальную стоимость. При равенстве стоимостей остаётся ранее выбранная операция: сначала -1, затем /2, затем /3.

2. Восстановление ответа. Начиная с n, программа добавляет op[n] в последовательность и переходит к числу после этой операции. Процесс повторяется до достижения единицы. Обращать последовательность не требуется: операции уже записываются в порядке выполнения.

Доказательство корректности

Докажем по индукции, что dp[i] хранит минимальную стоимость. База: dp[1] = 0, поскольку исходное число уже равно единице.

Пусть утверждение верно для всех чисел меньше i. Любое преобразование i начинается с одной из допустимых операций и приводит к числу j < i. Стоимость первой операции равна i. По предположению индукции оптимальная стоимость оставшейся части равна dp[j]. Следовательно, любое решение стоит не меньше i + min dp[j] по допустимым j.

Алгоритм перебирает все такие переходы и выбирает минимум. Выбранная операция вместе с оптимальным преобразованием j достигает этой стоимости. Значит, вычисленное dp[i] оптимально.

Массив op сохраняет переход, соответствующий найденному минимуму. Каждый переход строго уменьшает число и не опускает его ниже единицы. Поэтому восстановление завершается в единице, а сумма стоимостей выбранных операций равна dp[n].

Анализ сложности

Временная сложность — O(n): для каждого из n − 1 состояний проверяются не более трёх переходов. Восстановление и вывод L операций требуют O(L), где $L \le n - 1$, поэтому общая оценка остаётся O(n).

Пространственная сложность — O(n): таблица dp содержит n + 1 значений long long, таблица op — n + 1 значений перечисления с базовым типом char. Дополнительно хранятся L операций результата и строка вывода.

На использованной платформе основные таблицы занимают примерно 9(n + 1) байт. При $n = 10^7$ это около 85,8 МиБ без учёта остальных объектов и служебных расходов. В предыдущем запуске максимальный расход памяти процесса составил около 89 МиБ. Соответствие ограничению памяти зависит от лимита проверяющей системы, который в предоставленном условии не указан.

Для стоимости выбран тип long long. Допустимый путь из одних вычитаний имеет стоимость n(n + 1) / 2 − 1. При максимальном n эта величина равна 50 000 004 999 999 и помещается в 64-битное целое число. Следовательно, минимальная стоимость также представима этим типом.

Структура программы

MinCostTransformer отвечает за вычисление оптимальной стоимости и восстановление пути. Его вспомогательные методы apply и restorePath скрыты в private. Все таблицы создаются внутри transform, поэтому отдельные запуски не зависят друг от друга.

Operation задаёт допустимые виды операций, TransformationResult объединяет стоимость и последовательность. OperationFormatter переводит операции в требуемый текстовый формат. InputReader и ResultWriter работают с потоками ввода и вывода, переданными через конструкторы.

Application координирует чтение, вычисление и вывод. Такое разделение обязанностей изолирует алгоритм от представления результата и соответствует принципам инкапсуляции и единственной ответственности. Наследование для решения этой задачи не требуется.

Исходный код

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

enum class Operation : char {
    None = 0,
    Subtract = 1,
    DivideBy2 = 2,
    DivideBy3 = 3
};

struct TransformationResult {
    long long cost = 0;
    vector<Operation> operations;
};

class MinCostTransformer {
public:
    TransformationResult transform(int n) const {
        vector<long long> dp(n + 1, 0);
        vector<Operation> op(n + 1, Operation::None);

        for (int i = 2; i <= n; ++i) {
            long long best = dp[i - 1];
            Operation bestOp = Operation::Subtract;

            if (i % 2 == 0 && dp[i / 2] < best) {
                best = dp[i / 2];
                bestOp = Operation::DivideBy2;
            }
            if (i % 3 == 0 && dp[i / 3] < best) {
                best = dp[i / 3];
                bestOp = Operation::DivideBy3;
            }

            dp[i] = best + i;
            op[i] = bestOp;
        }

        TransformationResult result;
        result.cost = dp[n];
        result.operations = restorePath(n, op);
        return result;
    }

private:
    static int apply(int value, Operation o) {
        switch (o) {
            case Operation::Subtract:  return value - 1;
            case Operation::DivideBy2: return value / 2;
            case Operation::DivideBy3: return value / 3;
            default:                   return value;
        }
    }
```

```
    static vector<Operation> restorePath(int n, const vector<Operation>& op) {
        vector<Operation> path;
        for (int cur = n; cur > 1; cur = apply(cur, op[cur])) {
            path.push_back(op[cur]);
        }
        return path;
    }
};

class OperationFormatter {
public:
    static string toString(Operation o) {
        switch (o) {
            case Operation::Subtract:  return "-1";
            case Operation::DivideBy2: return "/2";
            case Operation::DivideBy3: return "/3";
            default:                   return "";
        }
    }

    static string format(const vector<Operation>& ops) {
        string s;
        for (size_t i = 0; i < ops.size(); ++i) {
            if (i) s += ' ';
            s += toString(ops[i]);
        }
        return s;
    }
};

class InputReader {
public:
    explicit InputReader(istream& in) : in_(in) {}

    bool readNumber(int& n) {
        return static_cast<bool>(in_ >> n);
    }

private:
    istream& in_;
};

class ResultWriter {
public:
    explicit ResultWriter(ostream& out) : out_(out) {}

    void write(const TransformationResult& r) {
        out_ << r.cost << "\n"
             << OperationFormatter::format(r.operations) << "\n";
    }

private:
    ostream& out_;
};

class Application {
public:
    Application(InputReader reader, ResultWriter writer, MinCostTransformer transformer)
```

```
        : reader_(reader), writer_(writer), transformer_(transformer) {}

    int run() {
        int n;
        if (!reader_.readNumber(n)) return 0;

        TransformationResult result = transformer_.transform(n);
        writer_.write(result);
        return 0;
    }

private:
    InputReader reader_;
    ResultWriter writer_;
    MinCostTransformer transformer_;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    InputReader reader(cin);
    ResultWriter writer(cout);
    MinCostTransformer transformer;

    Application app(reader, writer, transformer);
    return app.run();
}
```

Тесты

Результаты ниже получены запуском проверяемой программы. В каждом тесте стоимость равна сумме значений числа перед операциями. При равных стоимостях приведён путь, выбранный текущей реализацией.

Тест 1. n = 2

```
Ввод:
2
Вывод:
2
-1
```

Тест 2. n = 3

```
Ввод:
3
Вывод:
3
/3
```

Тест 3. n = 4

```
Ввод:
4
Вывод:
6
/2 -1
```

Тест 4. n = 5

```
Ввод:
5
Вывод:
11
-1 /2 -1
```

Тест 5. n = 6

```
Ввод:
6
Вывод:
8
/3 -1
```

Тест 6. n = 10

```
Ввод:
10
Вывод:
21
/2 -1 /2 -1
```

Тест 7. n = 27

```
Ввод:
27
Вывод:
39
/3 /3 /3
```

Тест 8. n = 82

```
Ввод:
82
Вывод:
202
-1 /3 /3 /3 /3
```

Проверка решения

В примере n = 82 программа выводит стоимость 202 и последовательность -1 / 3 /3 /3 /3. Числа перед операциями: 82, 81, 27, 9, 3; их сумма равна 202.

Для n = 2 выбирается -1, хотя /2 имеет ту же стоимость. Для n = 4 последовательность /2 -1 стоит 4 + 2 = 6. Это подтверждает, что при равенстве цен выбор операции не нарушает оптимальность.

Все входы от 2 до 1000 были сопоставлены с независимым решением через алгоритм Дейкстры: вершины графа — значения числа, рёбра — допустимые операции с весом, равным исходному числу. Во всех случаях минимальные стоимости совпали. Для каждого ответа дополнительно проверены допустимость делений, достижение единицы, сумма стоимости и двухстрочный формат вывода.

Проверены входы 9 999 999 и 10 000 000. Для максимального значения программа получила стоимость 20 079 033; выданная последовательность допустима, заканчивается в единице и имеет указанную стоимость. Эти большие тесты проверяют восстановление пути и работу на верхней границе; независимое сравнение оптимальности выполнено для диапазона 2…1000.

Сборка: g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -Wconversion main.cpp. Компилятор не выдал предупреждений. Измеренный ранее запуск для $n = 10^7$ занял около 0,07 с; это результат на текущем окружении, а не гарантированное время на другой машине.

Вывод

В ходе работы реализовано решение задачи минимизации стоимости преобразования натурального числа в единицу методом динамического программирования. Таблица оптимальных стоимостей позволяет учитывать все допустимые первые операции, а массив выбранных переходов — восстановить одну из оптимальных последовательностей.

Корректность решения обоснована индукцией по значению числа и подтверждена сравнением с независимым алгоритмом на малых входах. Реализация соответствует формату ввода и вывода и имеет линейную сложность по времени и памяти. Разделение вычислений, ввода и вывода между классами обеспечивает ясную структуру программы.